

**Instituição de Ensino: Proz Educação
Técnico em Desenvolvimento de Sistemas**

Nathália Renata , Thiago Junior, João Vitor Santana, Gabriel Gomes

ATIVIDADE AVALIATIVA - CRIPTOGRAFIA

**Cidade: Contagem - MG
Ano: 2025**

Sumário

1. Introdução.....	3
2. Distinções Conceituais: AES, MD5 E SHA-256.....	4
3. A Questão da Obsolescência: Algoritmos Inseguros.....	5
4. Desenvolvimento E Implementação.....	6
5. Criptografia Simétrica (O modelo "Chave única").....	7
5.1 Criptografia Assimétrica (Par de Chaves).....	8
6. Desafio Prático: Proteção de Arquivos.....	9
7. Conclusão	10
8. Referências Bibliográficas.....	11

INTRODUÇÃO

Na era da informação, a proteção de dados deixou de ser um diferencial para se tornar um requisito mandatório. Contudo, para quem está iniciando ou mesmo para profissionais da área, a "sopa de letrinhas" da criptografia pode causar confusão. Este trabalho tem como objetivo não apenas diferenciar as principais tecnologias utilizadas hoje — especificamente AES, MD5 e SHA-256 — mas também demonstrar, de forma prática e aplicada, como esses conceitos saem da teoria matemática e se transformam em linhas de código que protegem nossos arquivos diariamente.

DISTINÇÕES CONCEITUAIS: AES, MD5 E SHA-256

É comum haver confusão entre criptografia e hashing, pois ambos "embaralham" dados. No entanto, suas aplicações são drasticamente opostas.

O AES (Advanced Encryption Standard) é, de fato, um algoritmo de criptografia. Imagine-o como um cofre de alta segurança: você coloca o dado dentro, tranca com uma chave e, o mais importante, pode destrancar depois se tiver a cópia dessa chave. Ele é classificado como simétrico, pois a chave que fecha é a mesma que abre. É o padrão mundial para confidencialidade.

Por outro lado, o MD5 e o SHA-256 não são feitos para esconder informações reversíveis, mas para garantir integridade. Eles são funções de Hash. Pense neles como uma "impressão digital" digital. Se você passar um arquivo de 1GB por eles, receberá uma sequência pequena de caracteres. Se mudar uma vírgula no arquivo, essa sequência muda completamente. A diferença crucial entre os dois é a robustez: o MD5 é antigo e gera uma impressão digital curta (128 bits), enquanto o SHA-256 é moderno, gera uma impressão longa (256 bits) e é matematicamente muito mais complexo de falsificar.

Vulnerabilidade é corrigida através de patches ou atualizações de software/firmware.

Exemplos disso são um buffer overflow, uma senha padrão fraca, falta de validação de entrada de usuário em um formulário web.

Já o exploit tem como objetivo obter acesso, escalar privilégios, instalar um payload (carga maliciosa, como um ransomware ou backdoor) ou causar negação de serviço.

Tipos Comuns:

Exploit Remoto: É executado pela rede, sem acesso prévio ao sistema.

Exploit Local: Requer que o atacante já tenha algum acesso ao sistema, geralmente usado para elevar privilégios (ex: de um usuário comum para administrador)

Exploit Zero-Day (Dia Zero): Explora uma vulnerabilidade que é desconhecida pelos desenvolvedores do software, o que significa que não há correção (patch) disponível no momento do ataque.

A QUESTÃO DA OBSOLESCÊNCIA: ALGORITMOS INSEGUROS

A segurança é uma corrida de gato e rato. O que era seguro nos anos 90 hoje é trivialmente quebrado pelo poder computacional de um smartphone.

Atualmente, o MD5 (mencionado anteriormente) é considerado inseguro para uso em segurança crítica. Pesquisadores já provaram que é possível gerar dois arquivos diferentes com o mesmo hash MD5 (um ataque chamado de "colisão"). Outro exemplo clássico é o DES (Data Encryption Standard), o avô do AES. Sua chave é tão curta que computadores modernos conseguem testar todas as combinações possíveis em questão de horas.

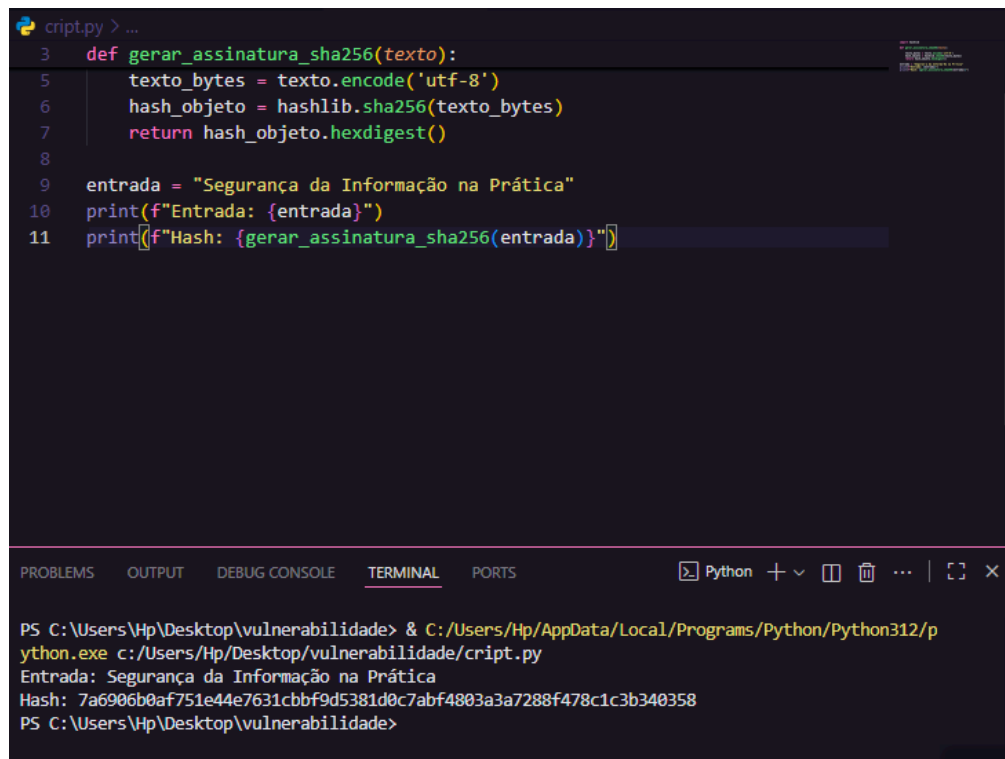
O uso desses algoritmos hoje é desencorajado, exceto para fins de compatibilidade com sistemas legados ou verificação simples de integridade onde não há risco de ataques maliciosos.

DESENVOLVIMENTO E IMPLEMENTAÇÃO

Para solidificar o entendimento teórico, apresento abaixo implementações funcionais utilizando a linguagem Python. Optou-se pelo uso da biblioteca cryptography, que é o padrão da indústria por oferecer primitivas seguras e testadas.

Verificação de Integridade (Hashing)

Neste primeiro exemplo, criamos uma função simples que recebe um texto qualquer e devolve sua assinatura SHA-256. Note que não há "chave", pois o hash é público.



The image shows a code editor with a Python script named `cript.py` and its execution output in a terminal window. The script defines a function `gerar_assinatura_sha256` that takes a text input, encodes it to bytes, and returns its SHA-256 hexadecimal digest. The script also includes a test case where the input is "Segurança da Informação na Prática". The terminal output shows the command to run the script, followed by the printed input and the resulting SHA-256 hash.

```
cript.py > ...
3 def gerar_assinatura_sha256(texto):
5     texto_bytes = texto.encode('utf-8')
6     hash_objeto = hashlib.sha256(texto_bytes)
7     return hash_objeto.hexdigest()
8
9 entrada = "Segurança da Informação na Prática"
10 print(f"Entrada: {entrada}")
11 print(f"Hash: {gerar_assinatura_sha256(entrada)}")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [X] [ ] [X]
PS C:\Users\Hp\Desktop\vulnerabilidade> & C:/Users/Hp/AppData/Local/Programs/Python/Python312/python.exe c:/Users/Hp/Desktop/vulnerabilidade/cript.py
Entrada: Segurança da Informação na Prática
Hash: 7a6906b0af751e44e7631cbbf9d5381d0c7abf4803a3a7288f478c1c3b340358
PS C:\Users\Hp\Desktop\vulnerabilidade>
```

Criptografia Simétrica (O modelo "Chave única")

Aqui utilizamos o algoritmo Fernet (baseado em AES). A lógica é simples: quem tem a chave faz tudo. Se você perder a chave, perde os dados para sempre. Se alguém roubar a chave, seus dados estão comprometidos

```

1 from cryptography.fernet import Fernet
2
3 def demonstracao_simetrica():
4     print("--- Iniciando Criptografia Simétrica ---")
5
6     chave = Fernet.generate_key()
7     ferramenta = Fernet(chave)
8
9     mensagem = b"Texto confidencial que nao deve ser lido."
10
11
12     token_cifrado = ferramenta.encrypt(mensagem)
13     print(f"Mensagem cifrada: {token_cifrado}")
14
15     mensagem_recuperada = ferramenta.decrypt(token_cifrado)
16     print(f"Mensagem original: {mensagem_recuperada.decode()}")
17
18 demonstracao_simetrica()

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

PS C:\Users\Hp\Desktop\vulnerabilidade> & C:/Users/Hp/AppData/Local/Programs/Python/Python312/python.exe
xe c:/Users/Hp/Desktop/vulnerabilidade/crypt2.py
--- Iniciando Criptografia Simétrica ---
Mensagem cifrada: b'gAAAAABpOKtyJ_7JC6Coj3bMMvV9icFI9ITAw9u0J591ACNp4mv80zFShs-cM_qqc1K0skzg6_1z3wJDD
Zz3aPEFVE6rMvCHqraCCAAy7Sd9T-4dqmKgid2TEdbu1EMQEJ5G0ELDfo'
Mensagem original: Texto confidencial que nao deve ser lido.
PS C:\Users\Hp\Desktop\vulnerabilidade>

```

Criptografia Assimétrica (Par de Chaves)

Este é o conceito mais fascinante. Eu gero duas chaves. Uma eu distribuo para o mundo (Pública) e a outra eu guardo a sete chaves (Privada). Qualquer pessoa pode usar minha chave pública para me mandar uma mensagem cifrada, mas apenas eu consigo ler com a privada. É assim que a internet segura (HTTPS) funciona.

```

1 from cryptography.hazmat.primitives.asymmetric import rsa, padding
2 from cryptography.hazmat.primitives import hashes
3
4 def demonstracao_assimetrica():
5     print("\n--- Iniciando Criptografia Assimétrica ---")
6
7     chave_privada = rsa.generate_private_key(
8         public_exponent=65537,
9         key_size=2048
10    )
11    chave_publica = chave_privada.public_key()
12
13    texto = b"Este dado viajou de forma segura."
14
15    cifrado = chave_publica.encrypt(
16        texto,
17        padding.OAEP(
18            mgf=padding.MGF1(algorithm=hashes.SHA256()),
19            algorithm=hashes.SHA256(),
20            label=None
21        )
22    )
23
24    decifrado = chave_privada.decrypt(
25        cifrado,
26        padding.OAEP(
27            mgf=padding.MGF1(algorithm=hashes.SHA256()),
28            algorithm=hashes.SHA256(),
29            label=None
30        )
31    )
32
33    print(f"Texto recuperado com sucesso: {decifrado.decode()}")
34
35 demonstracao_assimetrica()

```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Python + ▾ □ ☒ ... | 🔍

```

PS C:\Users\Hp\Desktop\vulnerabilidade> & C:/Users/Hp/AppData/Local/Programs/Python/Python312/python.exe c:/Users/Hp/Desktop/vulnerabilidade/cript2.py

--- Iniciando Criptografia Assimétrica ---
Texto recuperado com sucesso: Este dado viajou de forma segura.
PS C:\Users\Hp\Desktop\vulnerabilidade>

```


Desafio Prático: Proteção de Arquivos

Para finalizar, consolidei o conhecimento de criptografia simétrica em uma aplicação real. O algoritmo a seguir simula um comportamento de Ransomware (de forma ética e controlada) ou de um cofre digital: ele lê um arquivo do disco e o reescreve totalmente ilegível.

```
cript2.py > cifrar_arquivo_txt
1 import os
2 from cryptography.fernet import Fernet
3
4 def cifrar_arquivo_txt(caminho_arquivo):
5
6     if not os.path.exists(caminho_arquivo):
7         print("Erro: Arquivo não encontrado.")
8         return
9
10    chave = Fernet.generate_key()
11    fernet = Fernet(chave)
12
13    try:
14        with open(caminho_arquivo, 'rb') as file:
15            dados_originais = file.read()
16
17            dados_encryptados = fernet.encrypt(dados_originais)
18
19            novo_nome = caminho_arquivo + ".secreto"
20            with open(novo_nome, 'wb') as file:
21                file.write(dados_encryptados)
22
23            print(f"Sucesso! O arquivo '{novo_nome}' foi criado.")
24            print(f"ATENÇÃO - Guarde esta chave para decifrar: {chave.decode()}")
25
26    except Exception as e:
27        print(f"Ocorreu um erro durante o processo: {e}")
28
29    arquivo_teste = "relatorio_final.txt"
30    with open(arquivo_teste, "w") as f:
31        f.write("Conteúdo extremamente sensível do trabalho ABNT.")
32
33    cifrar_arquivo_txt(arquivo_teste)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
PS C:\Users\Hp\Desktop\vulnerabilidade> & C:/Users/Hp/AppData/Local/Programs/Python/Python312/python.exe c:/Users/Hp/Desktop/vulnerabilidade/cript2.py
Sucesso! O arquivo 'relatorio_final.txt.secreto' foi criado.
ATENÇÃO - Guarde esta chave para decifrar: f_dsNryj3bm9FyLTxCxh736x7fQ3Qaz5Ku80r7hUdqo=
PS C:\Users\Hp\Desktop\vulnerabilidade>
```

CONCLUSÃO

Conclui-se que a escolha entre AES, MD5 ou SHA-256 não é uma questão de qual é "melhor", mas sim de qual ferramenta resolve o problema proposto: sigilo ou integridade. As implementações demonstraram que, com poucas linhas de Python, é possível invocar algoritmos matemáticos complexos. Entretanto, a segurança real não reside apenas no algoritmo, mas na gestão correta das chaves criptográficas, que continua sendo o elo mais fraco da corrente de segurança.

REFERÊNCIAS BIBLIOGRÁFICAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: Informação e documentação — Trabalhos acadêmicos — Apresentação. Rio de Janeiro, 2011.

SCHNEIER, Bruce. **Applied Cryptography**: Protocols, Algorithms, and Source Code in C. 2. ed. John Wiley & Sons, 1996.

PYTHON SOFTWARE FOUNDATION. **Hashlib** — Secure hashes and message digests. Disponível em: <https://docs.python.org/3/library/hashlib.html>. Acesso em: 09 dez. 2025.